

Yazılım Mühendisliği Sonuç Raporu

KARADENİZ TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



PROJENİN KONUSU

2D metroidvania türü bilgisayar tabanlı macera odaklı, çocuk ve yetişkin herkesin oynayabileceği oyun geliştirmek.

HAZIRLAYANLAR

Alperen Altunel-425468

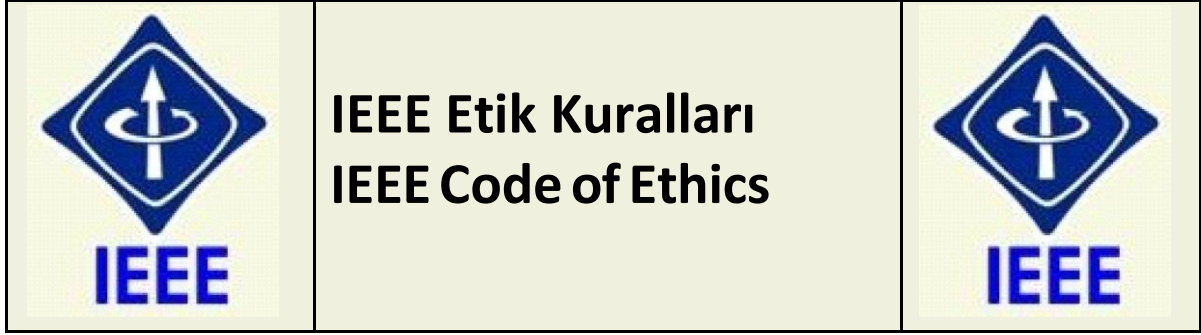
Berat Kayar-425500

Özer Koca-425472

Alaattin Uysal-425492

Eren Kuşdil-425429

2024-2025 BAHAR DÖNEMİ



Mesleğime karşı şahsi sorumluluğumu kabul ederek, hizmet ettiğim toplumlara ve üyelerine en yüksek etik ve mesleki davranışta bulunmaya söz verdiğimi ve aşağıdaki etik kurallarını kabul ettiğimi ifade ederim:

- Kamu güvenliği, sağlığı ve refahı ile uyumlu kararlar vermenin sorumluluğunu kabul etmek ve kamu veya çevreyi tehdit edebilecek faktörleri derhal açıklamak;
- Mümkün olabilecek çıkar çatışması, ister gerçekten var olması isterse sadece algı olması, durumlarından kaçınmak. Çıkar çatışması olması durumunda, etkilenen taraflara durumu bildirmek;
- Mevcut verilere dayalı tahminlerde ve fikir beyan etmelerde gerçekçi ve dürüst olmak;
- Her türlü rüşveti reddetmek;
- Mütenasip uygulamalarını ve muhtemel sonuçlarını gözeterek teknoloji anlayışını geliştirmek;
- Teknik yeterliliklerimizi sürdürmek ve geliştirmek, yeterli eğitim veya tecrübe olması veya işin zorluk sınırları ifade edilmesi durumunda ancak başkaları için teknolojik sorumlulukları üstlenmek;
- Teknik bir çalışma hakkında yansız bir eleştiri için uğraşmak, eleştiriyi kabul etmek ve eleştiriyi yapmak; hatları kabul etmek ve düzeltmek; diğer katkı sunanların emeklerini ifade etmek;
- Bütün kişilere adilane davranmak; ırk, din, cinsiyet, yaş, milliyet, cinsi tercih, cinsiyet kimliği, veya cinsiyet ifadesi üzerinden ayrımcılık yapma durumuna girişmemek;
- Yanlış veya kötü amaçlı eylemler sonucu kimsenin yaralanması, mülklerinin zarar görmesi, itibarlarının veya istihdamlarının zedelenmesi durumlarının oluşmasından kaçınmak;
- Meslektaşlara ve yardımcı personele mesleki gelişimlerinde yardımcı olmak ve onları desteklemek.

IEEE Yönetim Kurulu tarafından Ağustos 1990'da
onaylanmıştır

İÇİNDEKİLER

	Sayfa No
IEEE ETİK KURALLARI	2
İÇİNDEKİLER	3
1.Grup Bilgisi	4
2.Proje Konusu	4
3.Gereksinim Analizi	4
3.2. Gereksinim Toplama Yöntemleri	5
3.3. Gereksinimler (İşlevsel ve İşlevsel Olmayanlar)	6
3.4. Oyunun Müşteriye Sunumu: Pazarlama Stratejisi	8
3.4.1 Oyunun Benzersiz Değeri (Unique Selling Proposition-USP)	9
3.4.2. Hedef Kitle ve Pazar Analizi	9
3.4.3. Sunum Stratejisi	9
3.5. Gereksinimlerin Doğrulanması	10
3.6. Gereksinimlerin Yönetimi	11
3.7. Gereksinimlerin Önceliklendirilmesi	11
3.8. Kullanıcı Odaklı Yaklaşım	11
3.9. İteratif Yaklaşım	11
4.Mimari Tasarım	12
4.1.Sistemler	12
4.2. Alt	14
4.3. Sınıf Tanımları	15
4.4. Rutinler ve Arayüzler	19
4.5. Dil, Çerçeve ve Kodlama Standartları	20
4.6. Test Yaklaşımları	21
4.7.Mimari Diyagram	23
5.Sonuç	23

1. Grup Bilgisi

Ekip üyelerinden Alaattin Uysal. Görev dağılımını koordine eder, dokümantasyonu yönetir ve ekibin projeyi zamanında tamamlamasını sağlar. Ayrıca, oyunun müşteriye erişimini sağlar. Projenin zamanında tamamlanması, gereksinimlerin net bir şekilde belgelenmesi ve ekibin uyumlu çalışması.

Kimler: Berat Kayar, Eren Kuşdil, Alperen Altunel, Özer Koca, Alaattin Uysal

Rolleri: Her üye, oyunun belirli bir modülünü geliştirir:

Alaattin Uysal: Game Engine ve Menu (feature/engine).

Eren Kuşdil: Oyuncu Mekanikleri (feature/player).

Özer Koca: Düşmanlar ve Yapay Zekâ (feature/enemy).

Berat Kayar: Harita Tasarımı (feature/level).

Alperen Altunel: Arayüz tasarımı, sound manager (feature/player).

2. Proje Konusu

Bu proje, Karadeniz Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü öğrencileri olarak, yazılım mühendisliği dersimiz kapsamında 2D Metroidvania türünde bir oyun olan "The Way"i geliştirmek amacıyla başlatılmıştır. Amacımız hem eğlenceli hem de teknik açıdan yazılım mühendisliği prensiplerine uygun bir oyun ortaya koyarak ekip çalışması, proje yönetimi ve teknik becerilerimizi geliştirmektir. Proje sürecinde, gereksinim analizi, mimari tasarım ve uygulama aşamalarını titizlikle yürüttük. Bu rapor, projenin tüm aşamalarını sunmakta ve elde edilen sonuçları değerlendirmektedir.

3. Gereksinim Analizi

3.1. Paydaşlar

Projenin paydaşları, projeden doğrudan veya dolaylı olarak etkilenen veya projeye ilgi duyan taraflardır. The Way, bir oyun olarak tasarlanmış ve ilgili bir müşteri kitlesine pazarlanmak üzere geliştirilmiştir. Bu bağlamda, paydaşlar aşağıdaki gibi tanımlanmıştır:

- **Müşteriler (Oyun Yayıncıları ve Platformlar):**

- **İlgili kişi ve platformlar:** Bağımsız oyun yayıncıları, bilgisayar tabanlı dijital oyun platformları (Steam, Itch.io) veya oyun geliştirme yarışmalarını düzenleyen organizasyonlar.
 - **Amaç:** Oyunu satın alacak, yayınlayacak veya platformlarında dağıtacak ana paydaşlar. Bu grup, oyunun ticari değerini değerlendirir ve son kullanıcılara ulaştırır.
 - **Beklentiler:** Benzersiz bir oynanış deneyimi, görsel çekicilik, akıcı performans ve geniş bir kitleye hitap etme potansiyeli.
- **Son Kullanıcılar (Oyuncular):**

2D platform oyunlarını seven 13-30 yaş arası genç oyuncular, özellikle retro oyun meraklıları ve indie oyun hayranları, oyunun asıl hedef kitlesini oluşturuyor; bu kişiler oyunu oynayacak, deneyimleyecek ve sosyal medya ya da platform incelemeleri üzerinden geri bildirim sağlayacak, eğlenceli ve sürükleyici bir oynanış, sezgisel kontroller, görsel olarak çekici seviyeler ve tekrar oynanabilirlik gibi beklentilerle hareket edecekler.

Diğer İlgili Taraflar:

- **Kimler:** Yazılım mühendisliği dersi öğretim görevlisi ve asistanları, sınıf arkadaşları, oyun geliştirme toplulukları.
- **Rolleri:** Öğretim görevlisi ve asistanlar, projeyi değerlendirir ve geri bildirim sağlar. Sınıf arkadaşları, prototip aşamasında test yaparak geri bildirim verir. Oyun geliştirme toplulukları (örneğin, Reddit'teki indie oyun forumları), oyunun tanıtımı ve geri bildirim toplama sürecinde rol oynar.

3.2. Gereksinim Toplama Yöntemleri

Gereksinimleri toplamak için aşağıdaki yöntemler kullanılmıştır:

- **Görüşmeler:** Ekip, öğretim görevlisiyle görüşmeler yaparak projenin akademik beklentilerini anlamıştır. Ayrıca, potansiyel müşterilerle yapılan sanal görüşmelerle oyunun ticari gereksinimleri belirlenmiştir.
- **Beyin Fırtınası:** Ekip üyeleri, oyunun benzersiz özelliklerini belirlemek için her hafta beyin fırtınası oturumları düzenler.
- **Kullanım Senaryoları:** Oyuncu, oyunu başlatır ve sezgisel kontrollerle ana karakteri yönlendirir; seviyelerde zıplayarak, koşarak ve engelleri aşarak ilerler. Görsel olarak çekici tasarlanmış ortamlarda gizli ödülleri toplar, düşmanlarla karşılaşır ve basit ama eğlenceli mücadelelerle onları geçer. Her bölümün sonunda bir hedefe ulaşır ve ilerledikçe oyunun sürükleyici hikayesi ortaya

çıkar. Oyuncu, tekrar oynanabilirlik için farklı yolları dener, puan biriktirir ve deneyimlerini sosyal medyada paylaşır ya da inceleme yazar.

- **Prototipleme:** Basit bir ön model geliştirilerek, oyunun demo sürümü ekip üyeleri ve potansiyel oyuncular üzerinde test edilmiştir. Geri bildirimler, gereksinimlerin şekillenmesinde kullanılmıştır.

3.2.1. Açık ve Anlaşılır İletişim

- Ekip, müşterilerle iletişim kurarken oyunun değerini vurgulayan sunumlar hazırlamıştır.
- Ekip içi iletişim, yüz yüze ve sosyal mecralar üzerinden yürütülmüştür ve tüm gereksinimler net bir şekilde belgelenmiştir.
- Geri bildirimler, düzenli toplantılarda tartışılmış ve gereksinimlere yansıtılmıştır.

3.3. Gereksinimler

3.3.1 İşlevsel Gereksinimler

İşlevsel gereksinimler, sistemin ne yapması gerektiğini tanımlar.

GR-1: Oyun Motoru (Game Engine)

- **Açıklama:** Sistem, bir oyun motoru üzerinde çalışır.
- **Detaylar:**
 - Oyun döngüsü (game loop) bulunmalıdır.
 - Çarpışma algılama (collision detection) mekanizması olmalıdır.
 - Durum yönetimi desteklenmelidir (duraklatma, sıfırlama vb.).
 - **Öncelik:** Yüksek

GR-2: Oyuncu Mekanikleri

- **Açıklama:** Sistem, oyuncunun kontrol edilebilir bir karakter sunmalıdır.
- **Detaylar:**
 - Oyuncu, klavye ile hareket edebilmelidir.
 - Oyuncu, çevreyle etkileşim kurabilmelidir.
 - Oyuncunun can sistemi olmalıdır; belirli durumlar can seviyesini azaltır veya yükseltir.
 - **Öncelik:** Yüksek

GR-3: Düşmanlar ve Yapay Zekâ

- **Açıklama:** Sistem, düşman karakterleri ve yapay zekâ davranışları içermelidir.
- **Detaylar:**
 - Düşmanlar, oyuncuyu algılayıp takip edebilmeli, gerekli durumlarda platforma uygun hareket edebilmelidir.
 - Düşmanlar, belirli bir mesafeden oyuncuya saldırabilmelidir.
 - Düşman türleri farklı davranışlar sergilemelidir (devriye gezen düşmanlar, agresif düşmanlar vb.).
- **Öncelik:** Yüksek

GR-4: Seviye Tasarımı

- **Açıklama:** Sistem, birden fazla seviye ve platform içermelidir.
- **Detaylar:**
 - Her seviye, platformlar, engeller ve geçişler içermelidir.
 - Seviyeler arasında zorluk artışı olmalıdır.
 - Seviyeler arasında tema farkı olmalı ve kendi içlerinde temayı korumalıdır.
 - Seviye verileri, harici bir dosyadan (tmx) yüklenebilmelidir.
- **Öncelik:** Yüksek

GR-5: Dokümantasyon

- **Açıklama:** Sistem, kapsamlı bir dokümantasyon sağlamalıdır.
- **Detaylar:**
 - Oyun tasarımı dokümantasyonu (game_design.md) hazırlanmalıdır.
 - Görev dağılımı dokümantasyonu (team_roles.md) oluşturulmalıdır.
 - Hikâye dokümantasyonu (story.md) oluşturulmalıdır.
 - README dosyası, projenin genel yapısını ve kurulum adımlarını açıklamalıdır.
- **Öncelik:** Orta

3.3.2. İşlevsel Olmayan Gereksinimler

İşlevsel olmayan gereksinimler, sistemin çalışması için öncelikli olmayan ikincil

hedeflerdir

GR-6: Performans

- **Açıklama:** Oyun, düşük donanımlı sistemlerde akıcı bir şekilde çalışmalıdır.
- **Detaylar:** Oyun, en az 30 FPS (frame per second) hızında çalışmalıdır.
- **Öncelik:** Orta

GR-7: Kullanılabilirlik

- **Açıklama:** Oyun, kullanıcı dostu bir arayüze sahip olmalıdır.
- **Detaylar:**
 - Kontroller sezgisel olmalıdır (WASD ile hareket, Space ile zıplama).
 - UI elemanları, oyuncuyu rahatsız etmemelidir.
- **Öncelik:** Düşük

GR-8: Sürüm Kontrolü

- **Açıklama:** Proje, Git ve GitHub kullanılarak geliştirilmelidir.
- **Detaylar:**
 - Her modül için ayrı bir branch oluşturulmalıdır.
 - Değişiklikler, Pull Request (PR) ile dev dalına entegre edilmelidir.
- **Öncelik:** Yüksek

3.4. Oyunun Müşteriye Sunumu: Pazarlama Stratejisi

The Way, bir 2D platform oyunu olarak tasarlanmış ve müşterilere (oyun yayıncıları, dijital platformlar) sunulmak üzere geliştirilmektedir. Oyunu müşteriye en çekici şekilde sunmak için bir reklamcı/pazarlayıcı gibi düşünerek aşağıdaki stratejiler geliştirilmektedir.

3.4.1 Oyunun Benzersiz Değeri (Unique Selling Proposition-USP)

- **Nostaljik ama Modern:** The Way, retro 2D platform oyunlarının nostaljik hissini modern mekaniklerle birleştirmektedir. Sezgisel kontroller ve dinamik düşman yapay zekâsı, oyunculara hem tanıdık hem de yenilikçi bir deneyim sunacaktır.

- **Modüler Tasarım:** Oyunun modüler yapısı (ayrık seviyeler, farklı düşman türleri), genişletilebilirlik ve özelleştirme imkânı sunar. Böylelikle oyun, farklı platformlara veya kitlelere kolayca uyarlanabilmektedir.

3.4.2. Hedef Kitle ve Pazar Analizi

- **Hedef Kitle:** 13-30 yaş arası oyuncular, özellikle indie oyun meraklıları ve retro oyun hayranları. Bu kitle, Steam ve Itch.io gibi platformlarda aktif olarak oyun arar.
- **Pazar Fırsatları:** Indie oyun pazarı, son yıllarda büyük bir büyüme göstermiştir. Steam’de 2024’te yayınlanan indie oyunların %40’ı 100.000’den fazla indirme almıştır (SteamDB verilerine göre). The Way, bu pazarda kendine yer bulabilir.
- **Rakip Analizi:** Benzer oyunlar görsel estetik ve derin hikayelerle öne çıkıyor. The Way, daha basit fakat daha sürükleyici bir oynanış sunarak bu oyunlarla rekabet edebilir.

3.4.3. Sunum Stratejisi

Oyunu müşterilere sunarken aşağıdaki adımlar izlenecektir:

3.4.3.1. Görsel ve İşitsel Çekicilik

- **Demo Videosu:** Oyunun oynanışını gösteren 1-2 dakikalık bir demo videosu hazırlanacak. Video, oyuncunun bir seviyeyi geçtiği, düşmanlarla karşılaştığı ve hikâyede ilerlediği bir senaryo içerecektir. Video, enerjik bir müzikle desteklenecek.
- **Ekran Görüntüleri:** Oyunun en ilgi çekici seviyeleri ve karakterlerini içerecektir. Görseller, Steam veya Itch.io gibi platformlarda yer alacaktır.
- **Logo ve Marka Kimliği:** Oyunun logosu, oyunun nostaljik ve eğlenceli ruhunu yansıtacaktır. Bu logo, tüm pazarlama materyallerinde kullanılacaktır.

3.4.3.2 Sunum ve İletişim

- **Sunum Slaytları:** Müşterilere sunulmak üzere bir PowerPoint veya Canva sunumu hazırlanacak. Sunum, aşağıdaki bölümleri içerecek:
 - **Giriş:** Oyunun adı, logosu ve kısa bir tanıtım videosu.
 - **Benzersiz Özellikler:** Benzersiz karakterler, dinamik düşman yapay zekâsı ve modüler seviye tasarımı.
 - **Hedef Kitle:** Oyunun hangi oyunculara hitap ettiği ve pazar potansiyeli.
 - **Teknik Detaylar:** Kullanılan teknolojiler (Python, Pygame) ve performans (30 FPS).
 - **Sonraki Adımlar:** Oyunun yayınlanma süreci ve destek talepleri (pazarlama desteği).

- **E-posta İletimi:** Müşterilere gönderilecek bir tanıtım e-postası hazırlanacaktır.

3.4.3.3. Pazarlama ve Tanıtım

- **Sosyal Medya:** Oyunun tanıtımı için Twitter, Instagram ve Reddit'te paylaşımlar yapılacak. Kısa oynanış videoları ve ekran görüntüleri paylaşılacak.
- **Oyun Geliştirme Toplulukları:** Oyunun prototipi, oyun geliştirme topluluklarında paylaşılarak geri bildirim toplanacak ve ilgi çekilecek.
- **Oyun Yarışmaları:** Oyunun prototipi, indie oyun yarışmalarına gönderilerek görünürlük artırılacak.

3.4.3.4 Müşteriyle İş Birliği

- **Demo Sunumu:** Müşterilere canlı bir demo sunumu yapılacak. Sunumda, oyunun oynanışı, seviye tasarımı, karakter gelişimi vurgulanacak.
- **Esneklik:** Müşterinin taleplerine göre oyunun modüler yapısı sayesinde özelleştirmeler yapılabilir (iteratif yaklaşım).

3.5. Gereksinimlerin Doğrulanması

- **Doğrulama:** Gereksinimler, ekip üyeleri ve potansiyel müşterilerden alınan geri bildirimlerle doğrulanmıştır. Prototip üzerinden yapılan testler, gereksinimlerin doğru ve eksiksiz olduğunu göstermiştir. Gereksinimler, öğretim görevlisi tarafından onaylanacaktır.

3.6. Gereksinimlerin Yönetimi

Değişiklik Yönetimi Süreci

- **Değişiklik Talebi:** Müşterilerden veya son kullanıcılardan gelen geri bildirimler doğrultusunda gereksinim değişiklikleri önerilebilir.
- **Değerlendirme:** Değişiklik talebi, ekip toplantısında tartışılır ve etkileri (geliştirme süresi, maliyet) değerlendirilir.
- **Onay:** Değişiklik, ekip ve müşteri tarafından onaylanırsa uygulanır.

- **Belgeleme:** Onaylanan deęişiklik, gereksinim dokümanına eklenir ve ilgili paydaşlara bildirilir.

3.7. Gereksinimlerin Önceliklendirilmesi

Gereksinimler, müşteri ve son kullanıcı beklentilerine göre sıralanmıştır:

- **Yüksek Öncelikli Gereksinimler:** GR-1, GR-2, GR-3, GR-4, GR-8 (oyunun temel işlevsellięi ve kullanılabilirlięi için kritik).
- **Orta Öncelikli Gereksinimler:** GR-5, GR-6 (oyunun ek özelliklerini ve performansını iyileştirir).
- **Düşük Öncelikli Gereksinimler:** GR-7, müşteri taleplerine göre eklenebilir (örneğin, çok oyunculu mod).

3.8. Kullanıcı Odaklı Yaklaşım

- Gereksinimler, son kullanıcıyı (oyuncuları) merkeze alarak tanımlanmıştır. Örneğin:
 - Oyuncu mekanikleri (GR-3), sezgisel ve akıcı bir oynanış sağlamak için tasarlanmıştır.
- Prototip üzerinden alınan geri bildirimler, kullanıcı ihtiyaçlarını daha iyi anlamamızı sağlamıştır.

3.9. İteratif Yaklaşım

Gereksinim toplama süreci iteratif bir şekilde yürütölmüştür:

- İlk gereksinim seti, beyin fırtınası ve görüşmelerle oluşturulmuştur.
- Prototip geliştirildikten sonra, müşteriler ve çevrimiçi topluluklardan geri bildirim alınmış ve gereksinimler güncellenmiştir.

Müşteri sunumları öncesinde gereksinimler tekrar gözden geçirilecektir.

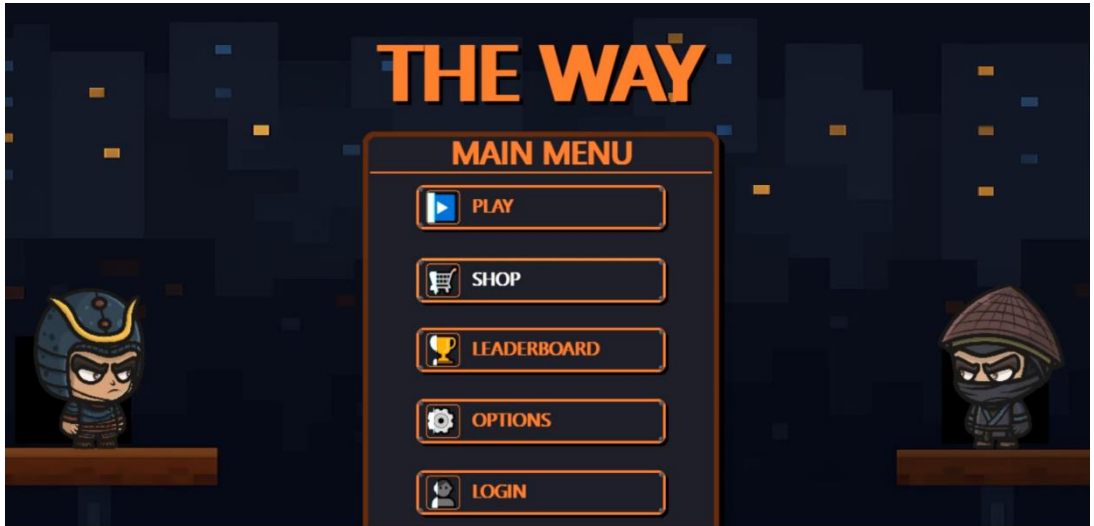
4.Mimari Tasarım

4.1.Sistemler

4.1.1 Menü Sistemi:

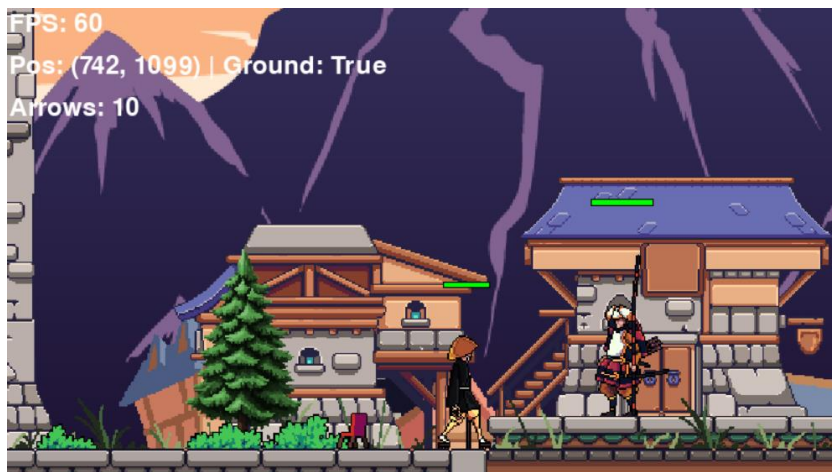
- Ana menü, ayarlar, alışveriş, seviye seçimi ve giriş ekranları dahil olmak üzere oyunun kullanıcı arayüzünü yönetir.

- Gezinti, ayar ayarlamaları (örneğin ses seviyesi, parlaklık) ve alışveriş işlemleri için kullanıcı girişlerini işler.
- Oyun oynanışına geçişi, oynanış sistemini başlatmak için sağlar.



4.1.2.Oynanış Sistemi:

- Oyuncunun hareketi, savaş, düşman yapay zekâsı ve harita işleme gibi temel oyun mekaniklerini uygular.
- Oyun içi etkileşimleri (örneğin iksir toplama, düşman karşılaşmaları, seviye geçişleri) yönetir.

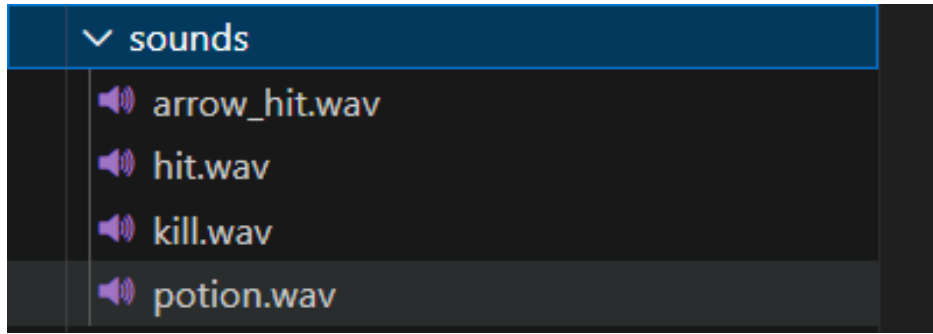


4.1.3. Durum Yönetimi Sistemi:

- Oyunun durumunu (örneğin oyuncu sağlığı, jetonlar, satın alınan yetenekler, ayarlar) merkezi bir şekilde yönetir ve sistemler arasında tutarlılık sağlar.
- Geçişler sırasında durumun kalıcılığı için seri hale getirme ve ters seri hale getirme sağlar.

4.1.4. Varlık Yönetimi Sistemi:

- Oyun varlıklarının (görüntüler, sesler, haritalar) yüklenmesini ve önbelleğe alınmasını yöneterek performansı optimize eder ve hata dayanıklılığını sağlar.

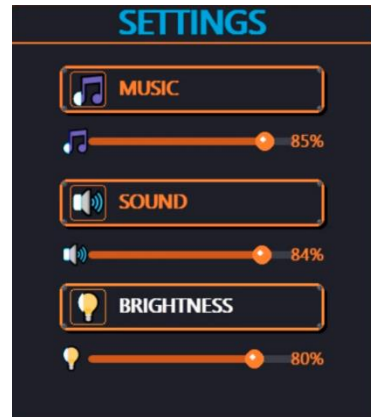
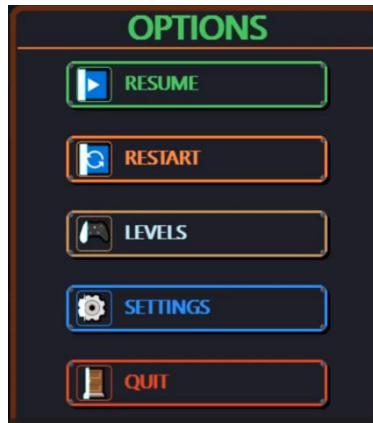


4.1.5. Oyun Yönetimi Sistemi:

- Menü ve oynanış sistemleri arasındaki geçişleri yönetir, uygun yaşam döngüsü yönetimi ve durum aktarımı sağlar.

4.1.6. Ayarlar Sistemi:

- Ses seviyesi, parlaklık, yakınlaştırma gibi oyun ayarlarını merkezi bir şekilde yönetir ve bunları menü ile oynanış sistemleri arasında uygular.



a4.2. Alt

Sistemler

Her sistem, modülerlik için alt sistemlere ayrılmıştır:

- Menü Sistemi:
 - UI İşleme Alt Sistemi: Düğmeler, kaydırmalı çubuklar, metin giriş kutuları ve arka plan görüntülerini işler.
 - Giriş İşleme Alt Sistemi: Gezinti ve etkileşimler için kullanıcı girişlerini işler.
 - Durum Geçiş Alt Sistemi: Oyun oynanışına geçişi, yönetim sistemiyle koordine eder.

Oynanış Sistemi:

- Oyuncu Alt Sistemi: Samurai sınıfını hareket, savaş ve sağlık için yönetir.

```
# Samurai (Player) class
class Samurai(pygame.sprite.Sprite):
    def __init__(self, walk_spritesheet, idle_spritesheet, jump_spritesheet, ...

    def update_grab_area(self):
        """Tutma alanını güncelle (karakterin üst kısmında bir alan)"""
        self.grab_area.midbottom = (self.hitbox.centerx, self.hitbox.top - 5)

    def check_wall_grab(self, collision_rects):
        """Duvarı tutup tırmanma kontrolü"""
        if self.is_climbing or self.is_hurt or self.is_dead or self.on_ground:
            return None
```

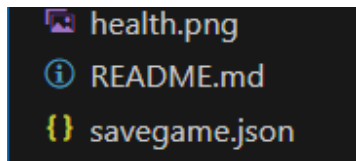
- Düşman Alt Sistemi: NinjaMonk için devriye, saldırı ve ölüm davranışlarını uygular.

```
# NinjaMonk sınıfı - Devriye gezen ve saldıran düşman
class NinjaMonk(pygame.sprite.Sprite):
    def __init__(self, idle_spritesheet, walk_spritesheet, attack_spritesheet, ...

    # Mevcut metodlar korunuyor, sadece devriye ve zıplama için yenileri ekleniyor
    def platform_kontrolu(self, collision_rects):
        """Platformun kenarında zıplama kontrolü"""
        simdiki_zaman = pygame.time.get_ticks()
        if simdiki_zaman - self.son_ziplama_zamani < self.ziplama_bekleme_suresi:
            return

        # Düşman hangi yöne bakıyorsa o yönde kenar kontrolü yap
        yon = -1 if self.flip else 1
```

- Harita Alt Sistemi: pytmx kullanarak harita yükleme, işleme, çarpışmalar ve geçişleri yönetir.
- Ses Alt Sistemi: Ses efektlerini ve arka plan müziğini yönetir.
- Durum Yönetimi Sistemi:
 - Seri Hale Getirme Alt Sistemi: Durumu JSON dosyalarına kaydeder ve yükler.
 - Durum Erişim Alt Sistemi: Durum değişkenleri için alıcılar ve ayarlayıcılar sağlar.



4.3. Sınıf Tanımları

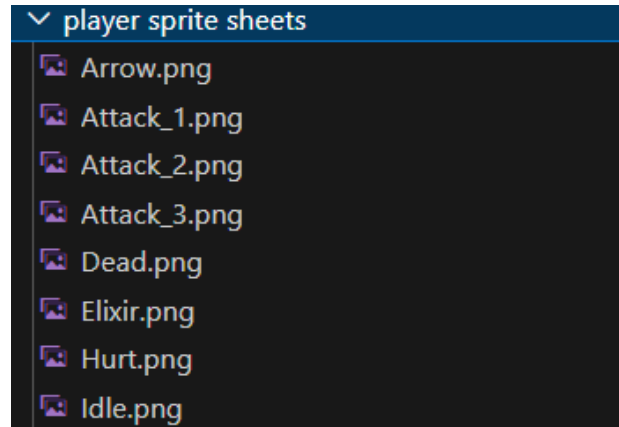
Aşağıda her sistem ve alt sistem için temel sınıflar, sorumlulukları ve öznitelikleri yer almaktadır.

- Durum Yönetimi Sistemi:
 - Sınıf: GameState
 - Öznitelikler:
 - state: Oyunun durumunu saklayan sözlük (örneğin, sağlık, jetonlar, satın alınan yetenekler).
 - state_file: Durumun kalıcılığı için JSON dosyasının yolu.
 - Yöntemler:
 - save(): Durumu JSON'a seri hale getirir.
 - load(): Durumu JSON'dan ters seri hale getirir.

Varlık Yönetimi Sistemi:

- Sınıf: AssetManager
 - Öznitelikler:
 - assets_dir: Varlıkların dizin yolu.
 - image_cache: Yüklenecek görüntülerin ön belleği.

- sound_cache: Yüklenen seslerin önbellesi.
- Yöntemler:
 - load_image(path, scale): Görüntüyü yükler ve önbellege alır, isteğe bağlı ölçeklendirme yapar.
 - load_sound(path): Ses dosyasını yükler ve önbellege alır



- Oyun Yönetimi Sistemi:
 - Sınıf: GameManager
 - Özellikler:
 - game_state: GameState örneği durum yönetimi için.
 - current_dir: Oyun dosyalarının dizin yolu.
 - Yöntemler:
 - start_game(): Durumu kaydeder ve player.py'yi başlatır.
 - return_to_menu(): Durumu kaydeder ve menu.py'yi başlatır.
- Ayarlar Sistemi:
 - Sınıf: Settings
 - Özellikler:
 - game_state: Ayarlara erişim için GameState örneği.
 - Yöntemler:
 - apply_settings(screen): Görsel ayarları ekrana uygular.
 - set_volume(music_volume, sound_volume): Ses seviyelerini ayarlar.

- Menü Sistemi:
 - Sınıf: Menu
 - Özellikler:
 - asset_manager: AssetManager örneği.
 - game_manager: GameManager örneği.
 - current_tokens: Oyuncu jetonlarını takip eder.
 - shop_skills: ShopSkill nesnelerinin listesi.
 - current_screen: Aktif menü ekranını takip eder.
 - Yöntemler:
 - load_background(): Menü arka planını yükler.
 - start_game(): GameManager ile oyun oynanışını başlatır.
 - update(): Kullanıcı girişine göre menü durumunu günceller.
 - Sınıf: PixelButton
 - Özellikler:
 - rect: Düğmenin konumu ve boyutu.
 - text: Düğme etiketi.
 - action: Tıklama için geri çağrı fonksiyonu.
 - Yöntemler:
 - update(mouse_pos, mouse_click): Düğme durumunu günceller (üzerinde gezinme, tıklama).
 - draw(screen): Düğmeyi işler.
- Oynanış Sistemi:
 - Sınıf: Samurai
 - Özellikler:
 - health, max_health: Oyuncu sağlık istatistikleri.
 - position: Oyuncunun konumu (x, y).
 - spritesheets: Yürüme, zıplama, saldırı gibi animasyonlar için sprite'lar.

- `attack_damage`: Saldırıların verdiği hasar.
- Yöntemler:
 - `move(keys)`: Hareketi işler (yürüme, koşma, zıplama).
 - `attack()`: Normal veya şarjlı saldırı yapar.
 - `shoot_arrow()`: Ok atma gibi ekstra işlevler.
 - `Check_hit_enemies`: Düşmana saldırma ve gerekli verilen zarar.

```

469 > def jump(self): ...
482
483 > def attack(self, attack_type): ...
503
504 > def shoot(self, arrow_group): ...
518
519 > def check_hit_enemies(self, enemies): ...
539

```

- Sınıf: NinjaMonk
 - Özellikler:
 - `health, max_health`: Düşman sağlık istatistikleri.
 - `position`: Düşmanın konumu (x, y).
 - `patrol_distance`: Devriye davranışı için mesafe.
 - `attack_damage`: Saldırının verdiği hasar.
 - Yöntemler:
 - `patrol()`: Belirli bir mesafede ileri geri hareket eder.
 - `attack(player)`: Oyuncuya saldırı yapar eğer mesafedeyse.
 - `update(player_pos)`: Oyuncu konumuna göre AI durumunu günceller.

```

1118 > def handle_collisions(self, dx, dy, collision_rects): ...
1143
1144 > def patrol(self, collision_rects): ...
1158
1159 > def update_attack_hitbox(self): ...
1164
1165 > def attack(self): ...
1173
1174     # Update the get_hit method:
1175 > def get_hit(self, damage): ...
1192
1193     # Update the detect_player method:
1194 > def detect_player(self, player): ...

```

- Sınıf: Map
 - Özellikler:
 - tmx_data: Yüklenen Tiled harita verileri.
 - collisions: Çarpışma nesnelerinin listesi.
 - hazards: Tehlike bölgelerinin listesi.
 - Transitions: Geçiş bölgelerinin listesi.
 - Yöntemler:
 - load_map(path): Tiled haritasını yükler.
 - draw(screen, camera): Paralaks kaydırmayla haritayı işler.
 - find_spawn_point(spawn_name="spawn_point"): Karakterin bölümde oluşacağı konumu bulur.

4.4. Rutinler ve Arayüzler

Aşağıda sistem etkileşimleri için temel rutinler ve arayüzler yer almaktadır:

- Durum Yönetimi Arayüzü:
 - get_state(key: str) -> Any: Durum değerini alır.
 - set_state(key: str, value: Any): Durum değerini günceller.
 - Uygulama: GameState sınıfı tarafından sağlanır.
- Varlık Yükleme Arayüzü:
 - load_asset(asset_type: str, path: str, *args) -> Any: Varlık yükler (görüntü veya ses).
 - Uygulama:

- `AssetManager.load_image(path, scale)` için görüntüler.
- `AssetManager.load_sound(path)` için sesler.
- Geçiş Arayüzü:
 - `transition_to(target: str)`: Başka bir sisteme geçiş yapar (menü veya oynanış).
 - Uygulama:
 - `GameManager.start_game()`: Oynanışa geçiş.
 - `GameManager.return_to_menu()`: Menüye dönüş.
- Ayarlar Arayüzü:
 - `apply_game_settings(screen: pygame.Surface)`: Görsel ayarları uygular.
 - `set_audio_settings(music_volume: float, sound_volume: float)`: Ses seviyelerini ayarlar.
 - Uygulama:
 - `Settings.apply_settings(screen)`: Parlaklık ayarı.
 - `Settings.set_volume(music_volume, sound_volume)`: Ses seviyeleri.
- Oynanış Güncelleme Rutini:
 - `update_gameplay(dt: float, keys: dict, events: list)`: Her çerçeveyi günceller.
 - Uygulama:
 - `player.py`'deki ana döngü, `Samurai`, `NinjaMonk` ve `Map` nesnelerini günceller:

4.5. Dil, Çerçeve ve Kodlama Standartları

- Dil: Python 3.10
 - Basitliği, geniş kütüphane desteği ve potansiyel web dağıtımı için Pyodide uyumluluğu nedeniyle seçilmiştir.
- Çerçeve: Pygame 2.6.1
- 2D işleme, ses yönetimi ve giriş yönetimi yetenekleri nedeniyle oyun geliştirme için kullanılmıştır.
- Harici Kütüphaneler:

- pytmx: Tiled haritalarını yüklemek ve işlemek için.
- pygame.mixer: Ses yönetimi için.
- JSON: durum değişimlerini kaydetmek ve uygulamak için.
- Kodlama Standartları:
 - Adlandırma Kuralları:
 - Sınıflar: CamelCase (örneğin, GameState, AssetManager).
 - Fonksiyonlar/Yöntemler: snake_case (örneğin, load_image, apply_settings).
 - Değişkenler: snake_case
 - Kod Yapısı:
 - Her modülün tek bir sorumluluğu olmalı.
 - Fonksiyon imzalarında tür ipuçları kullanılmalı.
 - Tüm sınıflar ve yöntemler için doküman dizeleri eklenmeli.
 - Hata Yönetimi:
 - Varlık yükleme ve dosya işlemleri için try-except blokları kullanılmalı.
 - Yükleme başarısız olursa yedek varlıklar (örneğin varsayılan görüntü) sağlanmalı.
 - Dosya Organizasyonu:
 - Tüm kaynak dosyalar src/ dizininde yer almalı.
 - Varlıklar assets/'te (örneğin, assets/menu_tasarim/, assets/sounds/).
 - Yapılandırma dosyaları (örneğin, game_state.json) kök dizinde.

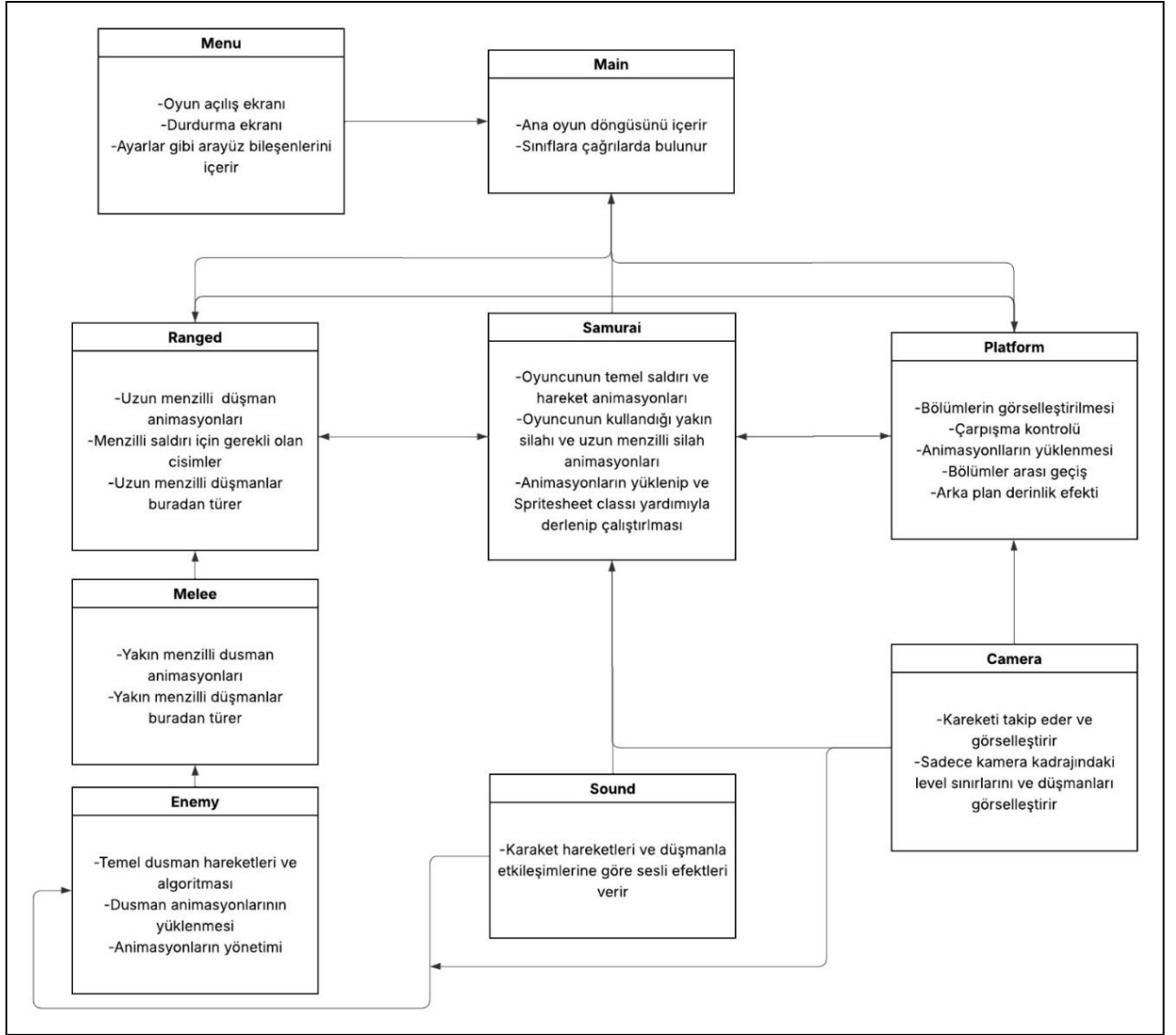
4.6. Test Yaklaşımları

Oyunun güvenilirliği ve performansı için testler kritiktir. Aşağıdaki yaklaşımlar kullanılacaktır:

- Birim Testleri:
 - Çerçeve: unittest
 - Odak:
 - GameState seri hale getirme/ters seri hale getirme testleri.

- AssetManager yedek davranış testi.
- Entegrasyon Testleri:
 - Odak:
 - menu.py ile player.py arasındaki geçişler için GameManager testi.
 - GameState'in sistemler arasında veri aktarımını doğru yapması.
- Fonksiyonel Testler:
 - Odak:
 - Oynanış mekanikleri testi.
 - Menü etkileşimlerini testi.
- Performans Testleri:
 - Odak:
 - Paralaks ön işleme ile ve olmadan kare hızı ölçümü.
 - AssetManager önbellekleme ile bellek kullanımını testi.
 - Araç: FPS ölçümü için pygame.time.Clock:
- Manuel Testler:
 - Kullanılabilirlik sorunlarını tanımlamak için oyunu oynatarak test et (örneğin menü gezintisi, oynanış tepkisi).
 - Kenar durumları test et (örneğin eksik varlıklar, geçersiz kullanıcı girişleri).

4.7.Mimari Diyagram



5.Sonuç

"The Way" projesi, yazılım mühendisliği prensiplerine uygun olarak modüler, ölçeklenebilir ve sürdürülebilir bir şekilde tamamlanmıştır. Gereksinim analizi, paydaşlarla iş birliği içinde gerçekleştirilmiş; MVC tabanlı mimari tasarım, modülerlik ve test edilebilirlik ilkelerine bağlı kalınarak uygulanmıştır. Proje, 12 haftalık süreçte iteratif bir yaklaşımla geliştirilmiş ve Git ile sürüm kontrolü sağlanmıştır. Oyun, hedef kitleye hitap edecek şekilde tasarlanmış ve indie oyun pazarında kendine yer bulma potansiyeline sahiptir. Proje sürecinde edindiğimiz en önemli tecrübe, ekip içi iletişimin ve dokümantasyonun önemi olmuştur. Gereksinimlerin net bir şekilde belgelenmesi ve düzenli geri bildirim döngüleri, projenin başarısını artırmıştır. Gelecekte, çok oyunculu mod gibi özellikler eklenerek oyunun genişletilmesi planlanmaktadır.